OS-ArchLab **ACADEMIC WORKSHOP & VIVA VOCE GUIDE**

OPERATING SYSTEMS HARDWARE ARCHITECTURE LAB

DMA Transfer Modes • CPU Instruction Execution Cycle • Memory Hierarchy & Cache Memory

Course: Operating System Concepts
Platform: OS-ArchLab Interactive Visualizer
Status: Evaluation & Examination Ready

PART 1: QUICKSTART & INTERFACE TOUR

OS-ArchLab is an interactive computer architecture and operating system hardware simulation platform. It bridges abstract low-level concepts into visible physical silicon events through three dedicated simulation engines:

- **Tab 1: DMA Modes Simulator:** Simulates Direct Memory Access controller (Intel 8237 style) cycle-by-cycle bus arbitration across Burst, Cycle Stealing, and Demand modes.
- **Tab 2: Instruction Cycle Lab:** Simulates the 3-phase machine cycle (Fetch, Decode, Execute) through an interactive clock pulse stepper across CPU registers (PC, MAR, MDR, IR, AC) and RAM.
- **Tab 3: Memory Hierarchy & Cache Lab:** Simulates the Speed Bridge, Multi-Level Cache (L1i/L1d, L2, L3, DRAM), Locality of Reference (Temporal & Spatial), and interactive Hit vs Miss block fills.

> **Launching & Running the Application**
> **React Web App:** Run `npm run dev` and open `http://localhost:5173` in Google Chrome or Edge.
> **Standalone Distribution:** Double-click `standalone.html` in any web browser with zero node/npm dependencies. Use the top navigation bar or URL hashes (`#dma`, `#instruction`, `#memory`) to switch labs. Press **F** or click **CINEMA MODE** for presentation view.

PART 2: DIRECT MEMORY ACCESS (DMA) & TRIPARTITE BUS

Direct Memory Access (DMA) is a specialized hardware mechanism that allows high-speed I/O peripherals (such as NVMe SSDs, network interfaces, and disk controllers) to transfer blocks of data directly to/from Main Memory (DRAM) without continuous CPU intervention.

The 4-Step Hardware Handshake Protocol

Step	Signal Line	Silicon Event & Bus State
Phase 1	`DREQ (DMA Request)`	I/O peripheral fills its hardware FIFO buffer and asserts DREQ high to the DMA Controller (8237).
Phase 2	`HOLD (Bus Request)`	DMA Controller asserts HOLD high to the CPU, requesting complete mastery of Address, Data, and Control buses.
Phase 3	`HLDA (Hold Ack)`	CPU completes current machine cycle, tri-states its bus drivers (high impedance), and returns HLDA=1 to DMA Controller.
Phase 4	`DACK & Data Transfer`	DMA Controller asserts DACK to peripheral, drives target RAM address onto Address Bus, and transfers data bytes at memory bus speed.

Comparison of the 3 DMA Transfer Modes

DMA Transfer Mode	Bus Holding Duration	CPU Interruption	Optimal Hardware Use-Case
Burst Mode (Block)	Holds bus continuously until entire block is transferred.	CPU completely paused (100% stalled) during transfer.	Fastest throughput for bulk transfers (e.g. disk sector / page reads).
Cycle Stealing Mode	Takes bus for 1 bus cycle per byte, then relinquishes.	Minimal CPU stall; CPU executes between steals.	Time-sensitive CPU processing concurrent with real-time sensors.
Demand Mode	Holds bus only while peripheral asserts DREQ active.	Intermittent CPU stall based on FIFO buffer state.	Streaming network packets, sound cards, bursty peripherals.

PART 3: CPU INSTRUCTION EXECUTION CYCLE & HARDWARE REGISTERS

The Central Processing Unit (CPU) executes programs through repeated iterations of the basic machine cycle composed of three distinct phases: **Fetch → Decode → Execute**.

Hardware Registers Cooperation

Register	Full Name	Architectural Role
PC	Program Counter	Holds the memory address of the next instruction to be fetched; auto-increments after fetch.
MAR	Memory Address Register	Holds the physical RAM address currently driven onto the Address Bus for read/write.
MDR	Memory Data Register	Holds the data word or instruction opcode received from or sent to RAM via Data Bus.
IR	Instruction Register	Holds the current instruction opcode while the Control Unit decodes and interprets it.
AC	Accumulator Register	Holds intermediate operands and results generated by the Arithmetic Logic Unit (ALU).

Step-by-Step Micro-Operation Walkthrough

The lab simulates the execution of the standard benchmark assembly program:

- `100: LOAD 500` — Load contents of Memory[500] (value **3456**) into Accumulator.
- `101: ADD 600` — Add contents of Memory[600] (value **1200**) to Accumulator (AC = **4656**).
- `102: STORE 700` — Write Accumulator (**4656**) into Memory[700].
- `103: HALT` — Stop CPU clock and terminate execution.

1. **Phase 1: Fetch Cycle (MAR ← PC, MDR ← Mem[MAR], IR ← MDR, PC ← PC + 1)**
   PC (100) is copied into MAR. The Control Unit asserts the `MEM_READ` line. Memory responds by placing "LOAD 500" into MDR. MDR contents copy to IR, and PC auto-increments to 101.

2. **Phase 2: Decode Cycle (Control Unit Opcode Resolution)**
   The Control Unit parses the opcode part of IR: identifies "LOAD" (RAM to AC copy) and extracts operand target address 500.

3. **Phase 3: Execute Cycle (ALU & Operand Transfer)**
   Address 500 is driven to MAR; `MEM_READ` asserted. Value **3456** enters MDR and transfers directly into Accumulator (AC = 3456).

PART 4: MEMORY HIERARCHY & CACHE MEMORY (THE SPEED BRIDGE)

CPU clock frequencies have outpaced physical DRAM access times by over 200×. Cache memory acts as an ultra-fast hardware buffer (Static RAM / SRAM) between CPU registers and DRAM, eliminating bus wait states.

Locality of Reference: The Core Operating System Principle

- **Temporal Locality:** If a memory location is accessed once, it is highly likely to be accessed again soon (e.g. loop index counters, stack frame variables, frequently called subroutines).
- **Spatial Locality:** When a memory location is accessed, nearby addresses will likely be accessed soon (e.g. linear array iteration, sequential instruction stream execution).

Multi-Level Cache Hierarchy (L1, L2, L3)

Cache Level	Physical Location	Typical Capacity	Latency & Architecture Role
L1 Cache	On-Core (Directly on CPU die)	32 KB – 128 KB	**1.0 ns (1–3 cycles):** Split into separate L1i (instruction) and L1d (data) caches to prevent fetch contention.
L2 Cache	On-Core / Dedicated Private	256 KB – 1 MB	**3.5 ns (10–14 cycles):** Services L1 cache misses; larger capacity buffer.
L3 Cache	Shared Processor Die	2 MB – 100 MB+	**12.0 ns (30–40 cycles):** Shared across all CPU cores; bridges on-chip caches to off-chip system bus.
Main Memory (RAM)	Motherboard System Bus	Gigabytes (8–64 GB)	**60.0 – 80.0 ns (150–200 cycles):** Slower Dynamic RAM (DRAM) working memory.

How Cache Works: Hits vs. Misses

- **Cache Hit:** When CPU references address, Tag matches line in SRAM. Data is delivered immediately in 1.0 ns with **zero DRAM bus traffic**.
- **Cache Miss:** If Tag mismatches (or Valid bit V=0), CPU suffers miss penalty. The memory controller fetches an entire **64-byte block** from DRAM into the cache line, populating data and setting Valid V=1.

PART 5: EVALUATOR VIVA VOCE QUESTIONS & HIGH-SCORING ANSWERS

Q1: Why is DMA faster than Interrupt-driven I/O or Programmed I/O?

Answer: In Programmed and Interrupt I/O, every individual byte/word must travel through the CPU accumulator register (requiring instructions: `IN`, `MOV`, `OUT`, plus context switching overhead). In DMA, the CPU delegates bus mastery to the DMA Controller; data flows directly from peripheral buffer to RAM across the system bus at memory bandwidth speeds without CPU instruction execution.

Q2: What is the difference between MAR and MDR during instruction fetch?

Answer: `MAR (Memory Address Register)` holds the address of the instruction currently being requested from RAM (connected to the unidirectional Address Bus). `MDR (Memory Data Register)` holds the binary opcode/data returned from RAM (connected to the bidirectional Data Bus). MAR specifies *where* to look; MDR stores *what* is fetched.

Q3: Why does the CPU auto-increment the Program Counter (PC) during the Fetch cycle instead of Execute?

Answer: Incrementing the PC during fetch allows the CPU to overlap address preparation with instruction decoding and execution (pipelining). If the instruction turns out to be a jump/branch, the execute phase will simply overwrite the PC with the target address.

Q4: Why is L1 cache split into separate L1i (Instruction) and L1d (Data) caches?

Answer: To support a Harvard microarchitecture on the core. Having separate instruction and data caches allows the CPU to fetch the next instruction (from L1i) at the exact same clock cycle that an executing instruction reads or writes an operand (to L1d), eliminating structural pipeline stalls.

Q5: What is Average Memory Access Time (AMAT) and how does cache improve it?

Answer: $\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$. Because modern L1/L2/L3 caches achieve a 90%–95% hit rate via Locality of Reference, the effective access time drops from 65.0 ns (pure DRAM) down to ~1.6–7.0 ns, providing a ~9× to 40× system speedup!
